

# **COMP0496 – Programação A**

## **Programação Orientada a Objetos — Parte 4**

Tópicos Avançados e Boas Práticas

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

@dataclass — Eliminando Boilerplate

@classmethod e @staticmethod

Princípios SOLID (Introdução)

Padrões de Projeto

Estudo de Caso — Refatorando o Podrão

Resumo do Módulo

# **@dataclass — Eliminando Boilerplate**

Uma classe simples com `__init__`, `__repr__` e `__eq__`:

```
1 class Lanche:
2     def __init__(self, nome, preco, ingredientes):
3         self.nome = nome
4         self.preco = preco
5         self.ingredientes = ingredientes
6
7     def __repr__(self):
8         return (f"Lanche(nome={self.nome!r}, "
9                 f"preco={self.preco!r}, "
10                f"ingredientes={self.ingredientes!r})")
11
12     def __eq__(self, other):
13         if not isinstance(other, Lanche):
14             return NotImplemented
15         return (self.nome == other.nome
16                 and self.preco == other.preco
17                 and self.ingredientes == other.ingredientes)
```

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Lanche:
5     nome: str
6     preco: float
7     ingredientes: list[str]
```

O decorador gera automaticamente:

- `__init__` — com todos os campos como parâmetros
- `__repr__` — representação legível
- `__eq__` — comparação campo a campo

## Resultado

3 linhas substituem ~20. O código expressa **o quê**, não **o como**.

```
1 @dataclass(frozen=True)
2 class ItemCardapio:
3     nome: str
4     preco: float
5     categoria: str
```

| Parâmetro   | Efeito                 | Quando usar                |
|-------------|------------------------|----------------------------|
| frozen=True | Imutável (sem setattr) | Itens de cardápio, configs |
| order=True  | Gera <, <=, >, >=      | Ordenar por campos         |
| eq=True     | Gera __eq__ (padrão)   | Quase sempre               |
| slots=True  | Usa __slots__ (3.10+)  | Performance                |

```
1 from dataclasses import dataclass, field
2
3 @dataclass
4 class Pedido:
5     cliente: str
6     itens: list[str] = field(default_factory=list)
7     observacoes: dict[str, str] = field(default_factory=dict)
8     numero: int = field(default=0, repr=False)
```

## Nunca faça isso

`itens: list[str] = []` — o mesmo objeto é compartilhado entre instâncias!

`field(default_factory=list)` cria uma **nova lista** para cada instância.

```
1 @dataclass
2 class Pedido:
3     cliente: str
4     itens: list[str] = field(default_factory=list)
5
6     def adicionar(self, item: str):
7         self.itens.append(item)
8
9     def total(self, cardapio: dict[str, float]) -> float:
10         return sum(cardapio[i] for i in self.itens)
11
12     def resumo(self) -> str:
13         return f"{self.cliente}: {len(self.itens)} itens"
```

## Ponto-chave

@dataclass gera o boilerplate; você adiciona a lógica de negócio.

## Use quando:

- Classe é principalmente “dados com métodos”
- Precisa de `__eq__` e `__repr__`
- Muitos campos (reduz boilerplate)
- Quer imutabilidade (`frozen`)

## NÃO use quando:

- `__init__` tem lógica complexa (validação pesada)
- Herança profunda (>2 níveis)
- Classe é majoritariamente comportamento
- Compatibilidade com Python <3.7

**@classmethod e @staticmethod**

| Tipo      | Decorador     | 1º Parâmetro | Acessa             |
|-----------|---------------|--------------|--------------------|
| Instância | (nenhum)      | self         | instância + classe |
| Classe    | @classmethod  | cls          | apenas classe      |
| Estático  | @staticmethod | (nenhum)     | nada da classe     |

## Regra prática

**Precisa de self?** → método de instância. **Precisa de cls?** → classmethod. **Não precisa de nenhum?** → staticmethod.

```
1 @dataclass
2 class Lanche:
3     nome: str
4     preco: float
5     categoria: str
6
7     @classmethod
8     def from_csv_line(cls, linha: str) -> "Lanche":
9         nome, preco, cat = linha.strip().split(",")
10        return cls(nome, float(preco), cat)
11
12    @classmethod
13    def from_dict(cls, d: dict) -> "Lanche":
14        return cls(**d)
```

```
1 x = Lanche.from_csv_line("X-Tudo,25.90,hamburguer")
2 y = Lanche.from_dict({"nome": "Suco", "preco": 8.0,
3                       "categoria": "bebida"})
```

## Função livre:

```
1 def lanche_from_csv(linha):
2     nome, preco, cat = \
3         linha.split(",")
4     return Lanche(
5         nome, float(preco), cat
6     )
```

- Funciona, mas fica “solta”
- Não herda: sempre cria Lanche

## @classmethod:

```
1 @classmethod
2 def from_csv_line(cls, linha):
3     nome, preco, cat = \
4         linha.split(",")
5     return cls(
6         nome, float(preco), cat
7     )
```

- Agrupada na classe
- cls cria o subtipo correto

## Herança

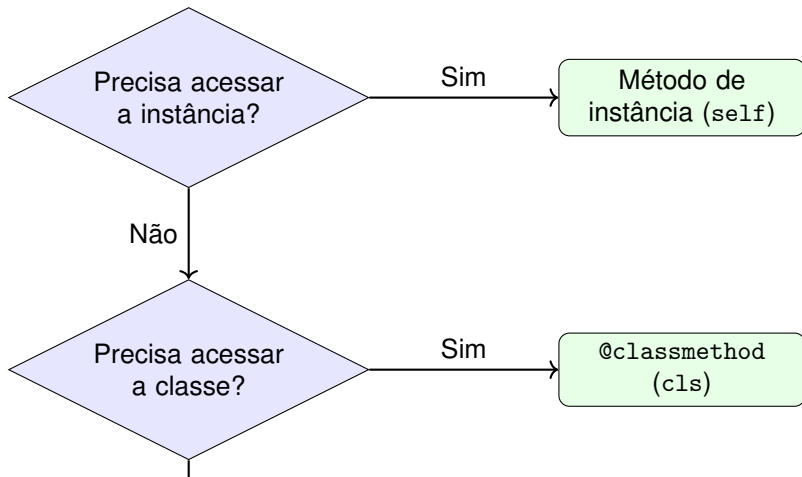
Se XTudo(Lanche), então XTudo.from\_csv\_line(...) retorna um XTudo, não um Lanche

```
1 class Lanche:
2     # ...
3
4     @staticmethod
5     def calcular_taxa_servico(valor: float,
6                               percentual: float = 0.10) -> float:
7         """Taxa de serviço sobre o valor."""
8         return round(valor * percentual, 2)
```

```
1 taxa = Lanche.calcular_taxa_servico(50.00) # 5.0
2 taxa = Lanche.calcular_taxa_servico(50.00, 0.15) # 7.5
```

## Quando usar

Lógica relacionada à classe, mas que não precisa de `self` nem de `cls`.  
Alternativa: função no módulo.



# Princípios SOLID (Introdução)

| Letra    | Princípio             | Ideia Central                                  |
|----------|-----------------------|------------------------------------------------|
| <b>S</b> | Single Responsibility | Uma classe, uma razão para mudar               |
| <b>O</b> | Open/Closed           | Aberta para extensão, fechada para modificação |
| <b>L</b> | Liskov Substitution   | Subtipos substituem o pai sem surpresas        |
| <b>I</b> | Interface Segregation | Interfaces pequenas e coesas                   |
| <b>D</b> | Dependency Inversion  | Dependa de abstrações, não de concretos        |

Hoje veremos **S**, **O** e **L** com exemplos do Podrão.

```
1 class PedidoRuim:
2     def __init__(self, cliente, itens):
3         self.cliente = cliente
4         self.itens = itens
5
6     def calcular_total(self): ... # lógica de negócio
7     def gerar_relatorio_html(self): ... # apresentação
8     def salvar_em_arquivo(self): ... # persistência
9     def enviar_email(self): ... # comunicação
```

## 4 razões para mudar

Mudança no cálculo, no formato HTML, no storage, ou no e-mail — todas alteram a mesma classe.

```
1 class Pedido:
2     def calcular_total(self): ...
3
4 class RelatorioHTML:
5     def gerar(self, pedido: Pedido): ...
6
7 class Armazenamento:
8     def salvar(self, pedido: Pedido): ...
9
10 class Enviador:
11     def enviar_email(self, pedido: Pedido, dest: str): ...
```

## Insight

Cada classe tem **uma única razão para mudar**. Testar cada uma é simples e independente.

```
1 class CalculadoraDesconto:
2     def calcular(self, pedido, tipo):
3         if tipo == "percentual":
4             return pedido.total() * 0.10
5         elif tipo == "fixo":
6             return 5.00
7         elif tipo == "fidelidade":
8             return pedido.total() * 0.15
9         # ... cada novo tipo exige elif
```

## Violação OCP

Para adicionar “combo”, precisa **modificar** CalculadoraDesconto. A classe não está fechada para modificação.

```
1 from abc import ABC, abstractmethod
2
3 class Desconto(ABC):
4     @abstractmethod
5     def calcular(self, total: float) -> float: ...
6
7 class DescontoPercentual(Desconto):
8     def __init__(self, taxa: float):
9         self.taxa = taxa
10    def calcular(self, total: float) -> float:
11        return total * self.taxa
12
13 class DescontoFixo(Desconto):
14     def __init__(self, valor: float):
15         self.valor = valor
16     def calcular(self, total: float) -> float:
17        return min(self.valor, total)
```

```
1 class Retangulo:
2     def __init__(self, largura, altura):
3         self._largura = largura
4         self._altura = altura
5
6     def set_largura(self, v):
7         self._largura = v
8
9     def area(self):
10        return self._largura * self._altura
11
12 class Quadrado(Retangulo):
13     def set_largura(self, v):
14         self._largura = v
15         self._altura = v # surpresa!
```

## Violação LSP

*“Se  $S$  é subtipo de  $T$ , objetos de  $T$  podem ser substituídos por objetos de  $S$  sem alterar as propriedades desejáveis do programa.”*

**Teste prático:** se o código cliente precisa checar `isinstance` para decidir o que fazer, provavelmente há violação de LSP.

### No Podrão

Hamburguer e XTudo são subtipos de Lanche. Qualquer código que recebe Lanche funciona com ambos — sem `if isinstance(...)`.

# Padrões de Projeto

```
1 class Pedido:
2     def calcular_desconto(self):
3         if self.tipo_desconto == "percentual":
4             return self.total * 0.10
5         elif self.tipo_desconto == "fixo":
6             return 5.00
7         elif self.tipo_desconto == "nenhum":
8             return 0.0
```

Já vimos isso no OCP. A solução: **Strategy Pattern**.

```
1 class SemDesconto(Desconto):
2     def calcular(self, total: float) -> float:
3         return 0.0
4
5 @dataclass
6 class Pedido:
7     cliente: str
8     itens: list[str] = field(default_factory=list)
9     desconto: Desconto = field(
10         default_factory=SemDesconto)
11
12     def total(self, cardapio: dict) -> float:
13         bruto = sum(cardapio[i] for i in self.itens)
14         return bruto - self.desconto.calcular(bruto)
```

## Composição

Pedido recebe a estratégia: não sabe qual é. Novo desconto – nova classe, zero

```
1 cardapio = {"X-Tudo": 25.90, "Suco": 8.00}
2
3 p1 = Pedido("Ana", desconto=DescontoPercentual(0.10))
4 p1.adicionar("X-Tudo")
5 p1.adicionar("Suco")
6 print(p1.total(cardapio)) # 30.51
7
8 p2 = Pedido("Bruno", desconto=DescontoFixo(5.00))
9 p2.adicionar("X-Tudo")
10 print(p2.total(cardapio)) # 20.90
```

A estratégia pode ser trocada em **tempo de execução**:

```
1 p1.desconto = DescontoFixo(10.00)
```

```
1 from typing import Callable
2
3 @dataclass
4 class PedidoFunc:
5     cliente: str
6     itens: list[str] = field(default_factory=list)
7     desconto: Callable[[float], float] = lambda t: 0.0
8
9     def total(self, cardapio: dict) -> float:
10         bruto = sum(cardapio[i] for i in self.itens)
11         return bruto - self.desconto(bruto)
```

```
1 p = PedidoFunc("Carlos",
2     desconto=lambda t: t * 0.10) # 10%
```

## Quando usar

Classes: quando a estratégia tem estado ou configuração. Funções: quando é

```
1 def criar_lanche(tipo: str):  
2     if tipo == "xtudo":  
3         return XTudo()  
4     elif tipo == "xsalada":  
5         return XSalada()  
6     elif tipo == "hamburguer":  
7         return Hamburguer()  
8     # ... cresce a cada novo lanche
```

## Problemas

Cadeia de if/elif viola OCP. Adicionar um tipo exige modificar a função.

```
1 class LancheFactory:
2     _registry: dict[str, type] = {}
3
4     @classmethod
5     def registrar(cls, nome: str, classe: type):
6         cls._registry[nome] = classe
7
8     @classmethod
9     def criar(cls, nome: str, **kwargs):
10         if nome not in cls._registry:
11             raise ValueError(f"Tipo desconhecido: {nome}")
12         return cls._registry[nome](**kwargs)
13
14 # Registro
15 LancheFactory.registrar("xtudo", XTudo)
16 LancheFactory.registrar("xsalada", XSalada)
17 LancheFactory.registrar("hamburguer", Hamburguer)
```

```
1  # Criação via string (ex: lido de arquivo/API)
2  lanche = LancheFactory.criar("xtudo", preco=25.90)
3
4  # Extensão em runtime --- sem tocar na Factory
5  class XBacon(Lanche):
6      def __init__(self, preco=28.00):
7          super().__init__("X-Bacon", preco, ["bacon"])
8
9  LancheFactory.registrar("xbacon", XBacon)
10 lanche2 = LancheFactory.criar("xbacon")
```

## Vantagens

- Desacopla criação de uso
- Extensível em runtime (registrar)
- String → objeto (útil para configs, APIs, CLIs)

# **Estudo de Caso — Refatorando o Podrão**

# O Código “Legado” (Procedural)



```
1  # Listas paralelas
2  nomes = ["X-Tudo", "X-Salada", "Suco"]
3  precos = [25.90, 20.00, 8.00]
4  quantidades = [0, 0, 0]
5
6  def fazer_pedido(idx, qtd):
7      quantidades[idx] += qtd
8
9  def calcular_total():
10     total = 0
11     for i in range(len(nomes)):
12         total += precos[i] * quantidades[i]
13     return total
14
15 def aplicar_desconto(total, tipo):
16     if tipo == "percentual": return total * 0.9
17     elif tipo == "fixo": return total - 5
18     return total
```

1. **Sopa de variáveis** — listas paralelas sem conexão explícita
2. **SRP violado** — tudo no mesmo escopo global
3. **OCP violado** — novo desconto = novo `elif`
4. **Sem encapsulamento** — qualquer parte do código altera quantidades
5. **Impossível testar** — funções dependem de estado global
6. **Impossível estender** — novo lanche = alterar 3 listas

## Diagnóstico

Este código funciona para 3 lanches. Com 30 lanches, 5 descontos e 3 formas de pagamento, vira pesadelo.

```
1 from abc import ABC, abstractmethod
2 from dataclasses import dataclass, field
3
4 class Lanche(ABC):
5     @abstractmethod
6     def preco(self) -> float: ...
7     @abstractmethod
8     def descricao(self) -> str: ...
9
10 @dataclass(frozen=True)
11 class Hamburguer(Lanche):
12     nome: str = "Hambúrguer"
13     _preco: float = 18.00
14     def preco(self) -> float: return self._preco
15     def descricao(self) -> str: return self.nome
16
17 @dataclass(frozen=True)
18 class XTudo(Lanche):
```

```
1 @dataclass
2 class Pedido:
3     cliente: str
4     itens: list[Lanche] = field(default_factory=list)
5     desconto: Desconto = field(
6         default_factory=SemDesconto)
7
8     def adicionar(self, lanche: Lanche):
9         self.itens.append(lanche)
10
11     def total(self) -> float:
12         bruto = sum(i.preco() for i in self.itens)
13         return bruto - self.desconto.calcular(bruto)
```

```
1 class Cozinha:
2     def processar(self, pedido: Pedido): ...
3
4 class Caixa:
```

| Conceito            | Aula | No Podrão                         |
|---------------------|------|-----------------------------------|
| Classe e objeto     | 1    | Lanche, Pedido                    |
| Atributos e métodos | 1    | nome, preco(), adicionar()        |
| Encapsulamento      | 1    | _preco, @property                 |
| Herança             | 2    | Hamburguer(Lanche), XTudo(Lanche) |
| Classe abstrata     | 2    | Lanche(ABC), Desconto(ABC)        |
| Polimorfismo        | 2    | preco() em cada subtipo           |
| Composição          | 3    | Pedido tem lista de Lanche        |
| @dataclass          | 4    | Pedido, ItemCardapio              |
| @classmethod        | 4    | Lanche.from_csv_line()            |
| Strategy            | 4    | Desconto → DescontoPercentual     |
| Factory             | 4    | LancheFactory.criar()             |
| SRP                 | 4    | Cozinha, Caixa, Pedido            |
| OCP                 | 4    | Novo desconto = nova classe       |

# Resumo do Módulo

| Aula | Tema                   | Conteúdo Principal                                                                                                  |
|------|------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1    | Classes e Objetos      | Classe, instância, atributos, métodos, <code>__init__</code> , encapsulamento, <code>@property</code>               |
| 2    | Herança e Polimorfismo | Herança, <code>super()</code> , ABC, <code>@abstractmethod</code> , <code>isinstance</code> , MRO                   |
| 3    | Composição e Agregação | “tem-um” vs “é-um”, composição, agregação, delegação, <code>__contains__</code>                                     |
| 4    | Avançado e SOLID       | <code>@dataclass</code> , <code>@classmethod</code> , <code>@staticmethod</code> , SRP, OCP, LSP, Strategy, Factory |

## 1. **Objetos agrupam dados e comportamento**

Não use listas paralelas. Crie uma classe que una o dado ao que se faz com ele.

## 2. **Prefira composição a herança**

Herança cria acoplamento forte. Composição permite trocar comportamento em runtime (Strategy).

## 3. **Cada classe, uma responsabilidade + extensão sem modificação**

SRP + OCP. Classes pequenas, coesas, extensíveis via polimorfismo.

**Se lembrar apenas disso...**

Estes três princípios guiam 90% das decisões de design em POO.

## Nesta disciplina:

- Testes unitários (pytest)
- Type hints e mypy
- Tratamento de exceções com classes
- Iteradores e geradores

## Em disciplinas futuras:

- Interface Segregation (I)
- Dependency Inversion (D)
- Mais padrões: Observer, Decorator, Adapter
- Arquitetura de software

**Obrigado! Dúvidas?**

andreyoshiaki@dcomp.ufs.br